



Wydział
Biologii

Wprowadzenie do głębokich architektur sieci neuronowych

Sieci rekurencyjne RNN

Wykład nr 5

dr Mateusz Draga

Zakład Hydrobiologii

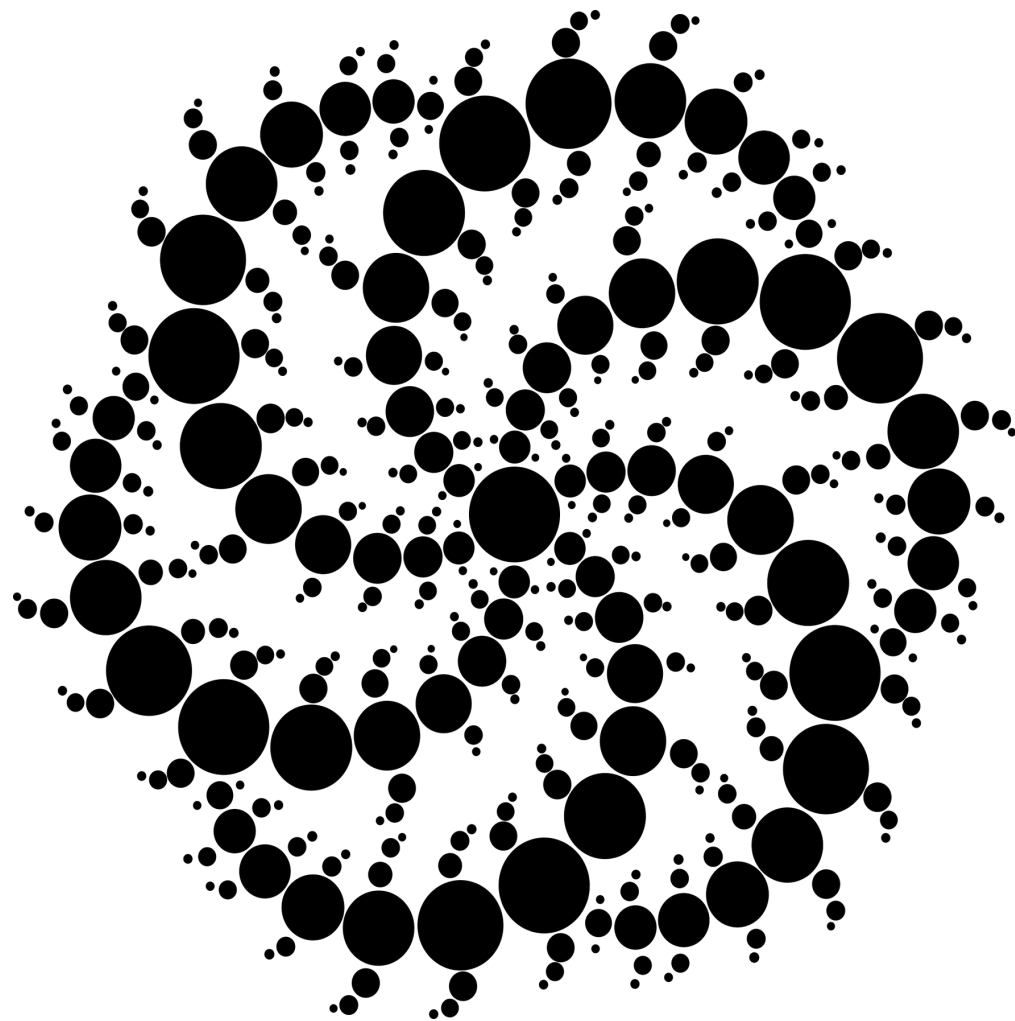
matdra@amu.edu.pl

Sieci RNN (Recurrent Neural Networks) to rodzaj sztucznych sieci neuronowych przeznaczonych do przetwarzania danych sekwencyjnych, takich jak tekst, mowa czy szeregi czasowe. Ich charakterystyczną cechą jest posiadanie pamięci wewnętrznej, dzięki której mogą uwzględniać informacje z poprzednich kroków sekwencji podczas analizy bieżących danych. RNN są wykorzystywane m.in. w rozpoznawaniu mowy, tłumaczeniu maszynowym, analizie języka naturalnego oraz prognozowaniu wartości w czasie. Dzięki zdolności modelowania zależności czasowych są szczególnie przydatne wszędzie tam, gdzie kolejność danych ma znaczenie.

Sieci typu Feed-Forward, takie jak ANN i CNN, nie posiadają mechanizmu pamięci umożliwiającego uwzględnianie poprzednich wejść i zazwyczaj przetwarzają dane o ustalonym rozmiarze. Natomiast sieci RNN są w stanie analizować dane sekwencyjne, przetwarzając kolejne elementy sekwencji jedna po drugiej, wykorzystując stan wewnętrzny do przechowywania informacji o wcześniejszych elementach.

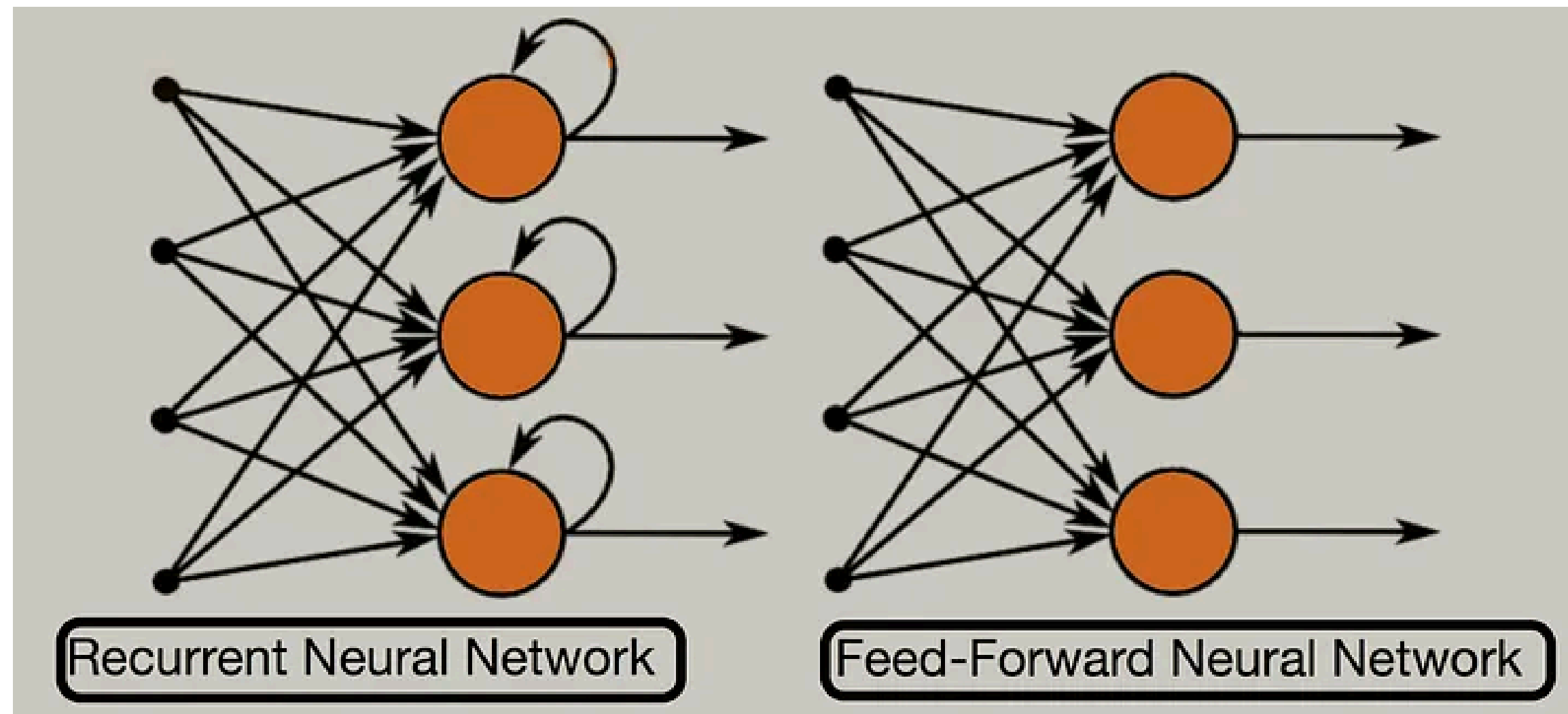


... the ground or stays
iverse is vast, and you
; also beautiful. You a
nething bigger than yc
t of something that ma
most of your time. Tal
e a blog post. Make a



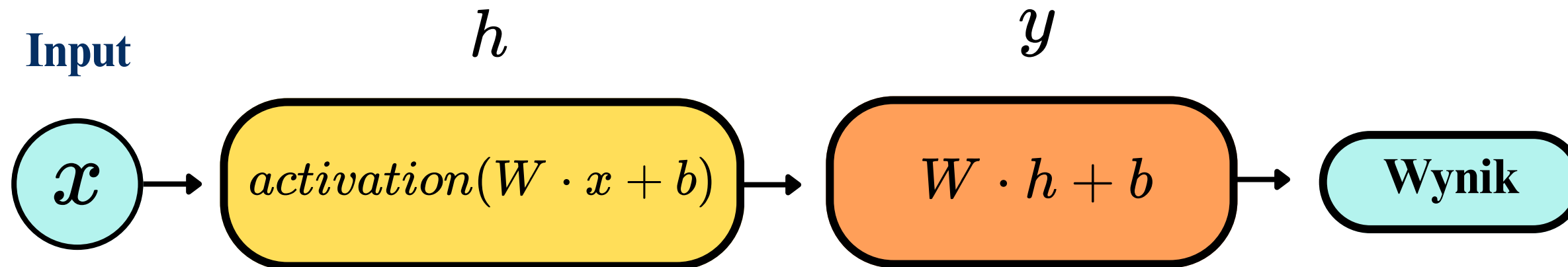
W zwykłej sieci neuronowej każdy input jest przetwarzany niezależnie. W RNN wyjście z poprzedniego kroku jest przekazywane z powrotem jako dodatkowy input do następnego kroku – stąd właśnie "rekurencja". Dokładniej mówiąc, do kolejnego kroku przekazywany jest stan ukryty (hidden state), który zawiera informacje z poprzednich kroków. To właśnie pozwala sieci „pamiętać” kontekst.

RNN to poprzednicy dzisiejszych modeli językowych. Mają one jednak problem z m.in. zanikającym gradientem, szczególnie przy dłuższych sekwencjach. Są też stosunkowo skomplikowane obliczeniowo. Dla tego aktualnie do modeli językowych częściej stosowane są inne typy sieci głębokich np. Transformery (GPT, BERT itd.)

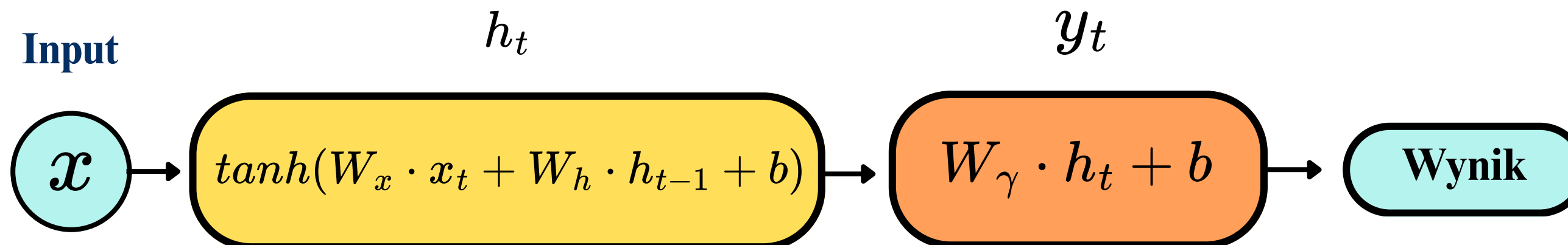


Źródło: Sachin Soni, Recurrent Neural Networks (RNN) from Basic to Advanced. Medium, 25.03.2024, dostęp: 03.05.2026, <https://medium.com/@nemagan/a-beginners-guide-to-neural-networks-and-deep-learning-6b5509526dd9>

Neuron w ANN:



Neuron w RNN:



h_t - nowy hidden state

x_t - aktualny input (np. słowo)

W_x - macierze wag wejściowych

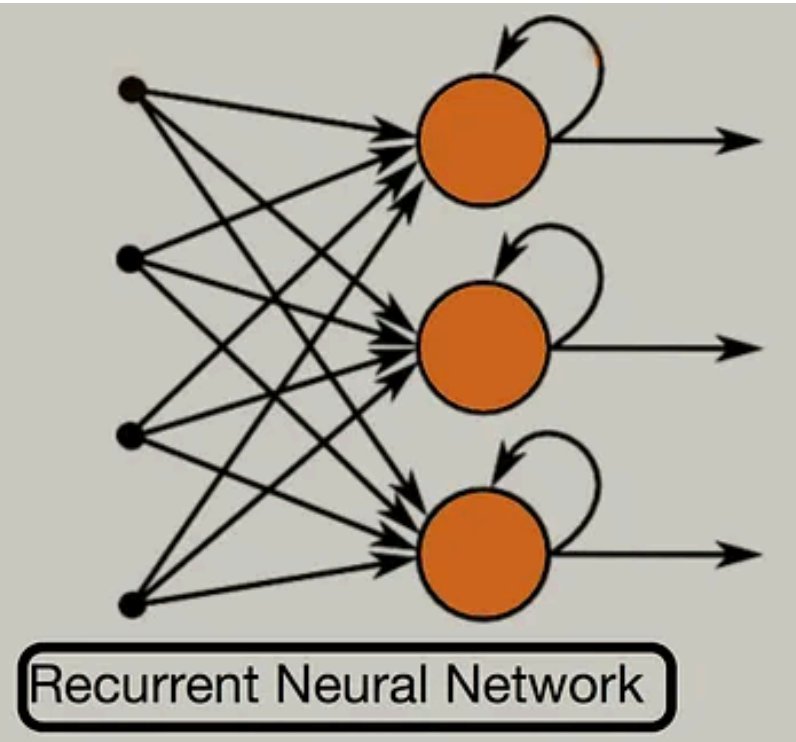
W_h - macierz wag rekurencyjnych

W_γ - macierze wag wyjściowych

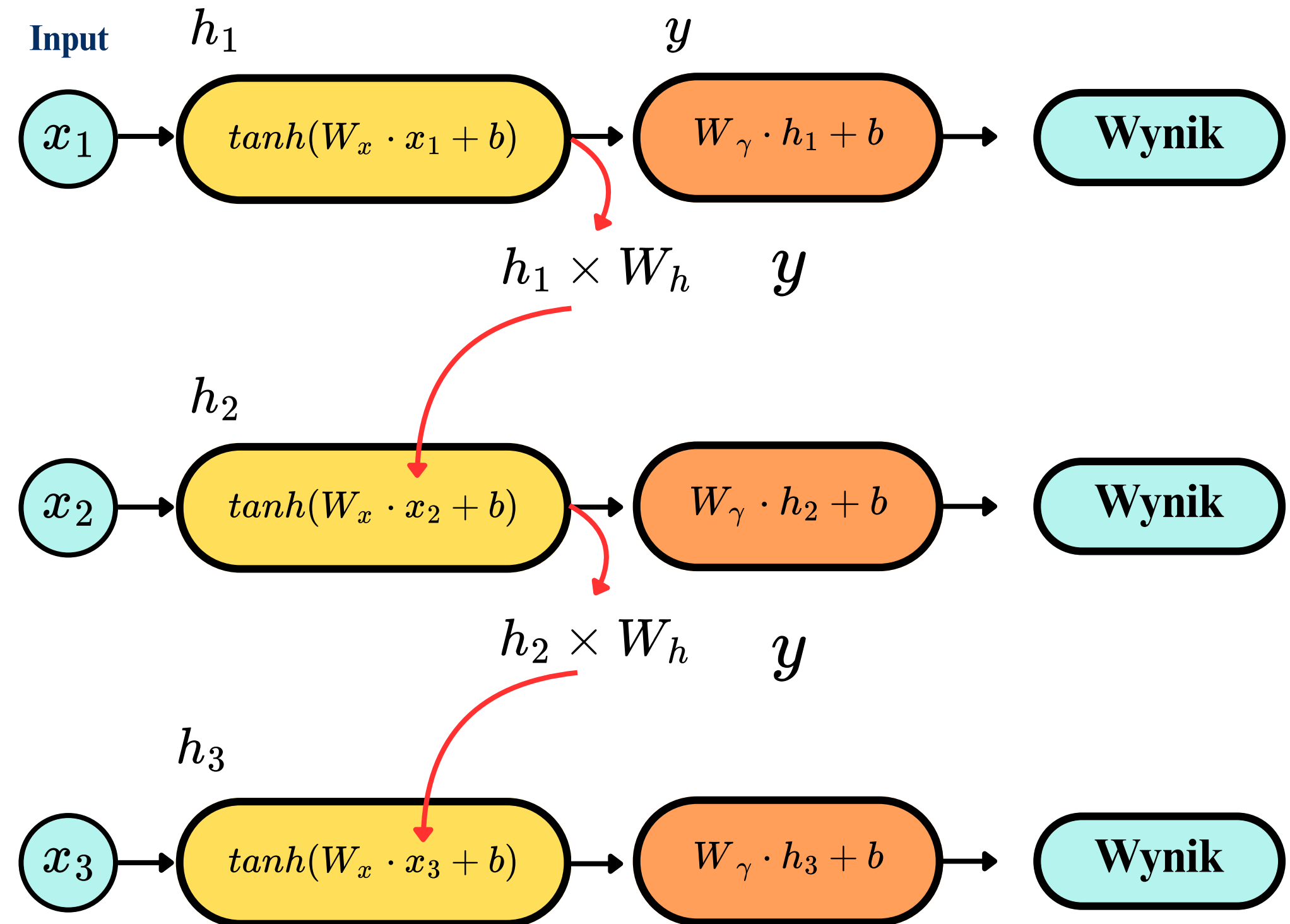
h_{t-1} - hidden state z poprzedniego kroku

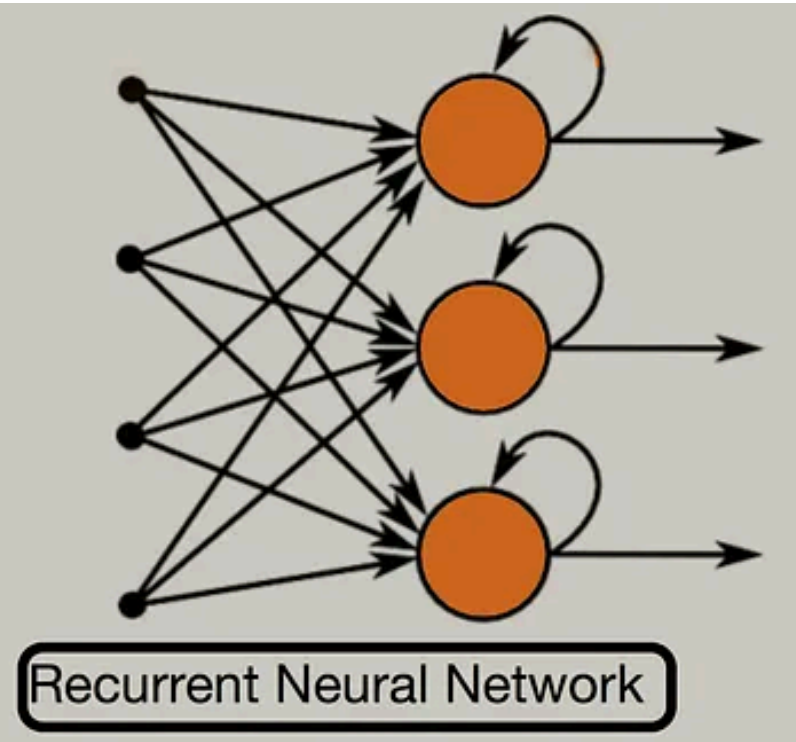
$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

$$y_t = W_\gamma \cdot h_t + b$$

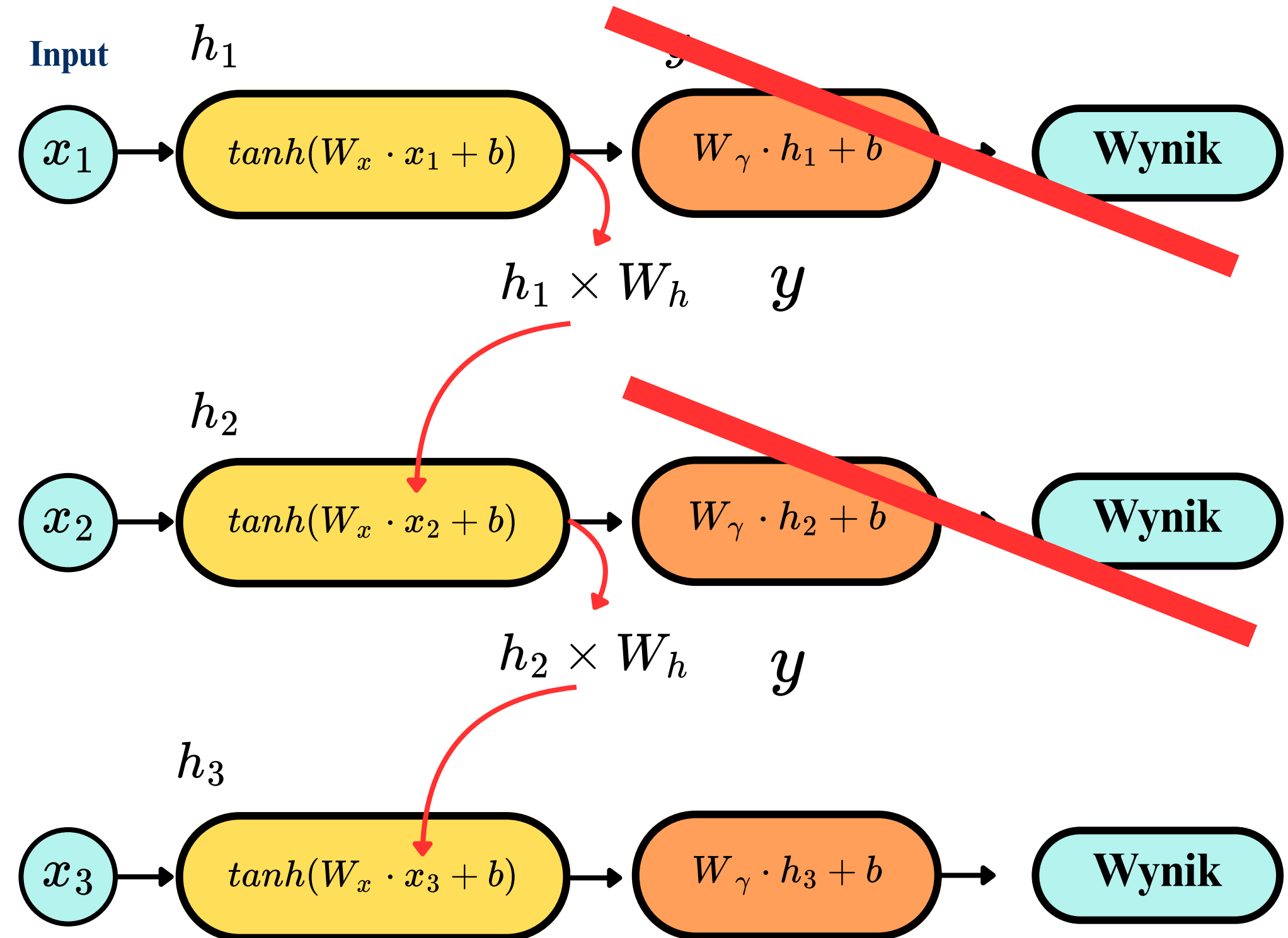


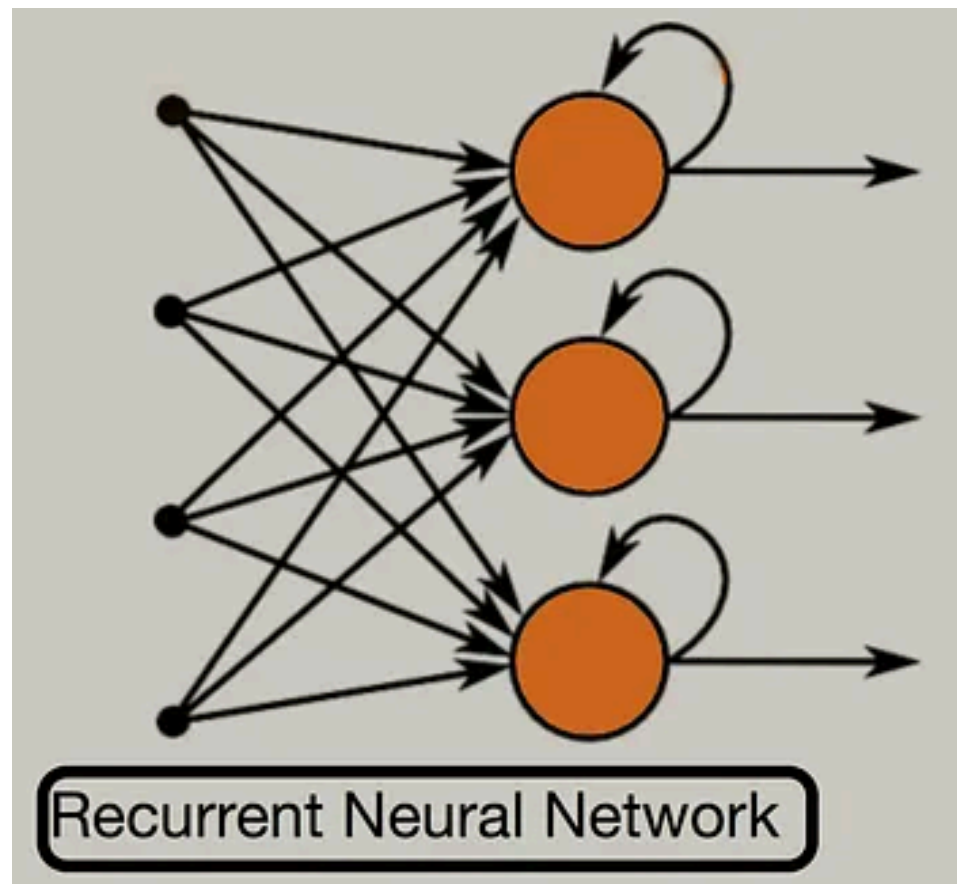
Hidden state to wektor liczb reprezentujący "pamięć" sieci w danym kroku czasowym, zakodowaną na podstawie wszystkich poprzednich inputów. Jest on przekazywany do następnego kroku jako dodatkowy input, co umożliwia sieci uwzględnianie kontekstu z wcześniejszych elementów sekwencji.





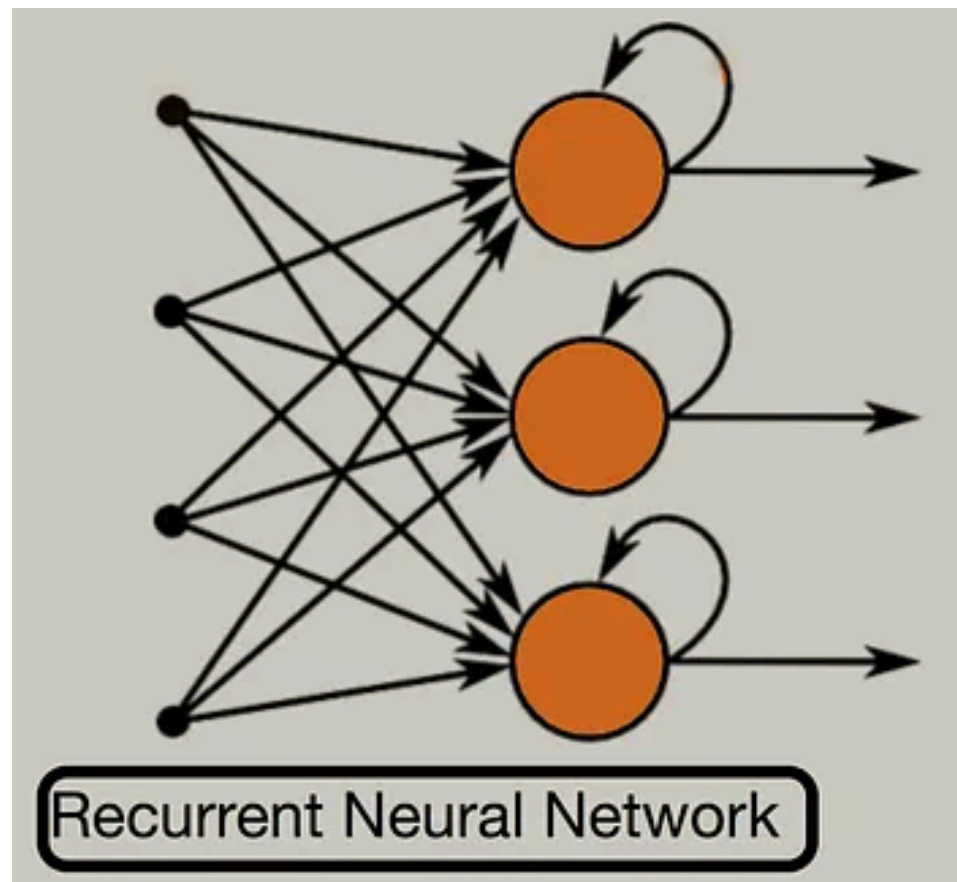
Hidden state to wektor liczb reprezentujący "pamięć" sieci w danym kroku czasowym, zakodowaną na podstawie wszystkich poprzednich inputów. Jest on przekazywany do następnego kroku jako dodatkowy input, co umożliwia sieci uwzględnianie kontekstu z wcześniejszych elementów sekwencji.





```
h = zeros(hidden_size) # stan początkowy = same zera

for x in sequence:
    h = tanh(Wx @ x + Wh @ h + b) # update stanu
    y = Wy @ h                    # output (opcjonalny)
```

```
h = zeros(hidden_size) # stan początkowy = same zera

for x in sequence:
    h = tanh(Wx @ x + Wh @ h + b) # update stanu
    y = Wy @ h                    # output (opcjonalny)
```

Text	Result
cat mat rat	1
rat rat mat	1
mat mat cat	0

Text	Result
[1,0,0] [0,1,0] [0,0,1]	1
[0,0,1] [0,0,1] [0,1,0]	1
[0,1,0] [0,1,0] [1,0,0]	0

- W_x - macierze wag wejściowych
- W_h - macierz wag rekurencyjnych
- W_γ - macierze wag wyjściowych

Współdzielenie parametrów w RNN polega na tym, że te same macierze wag W_x , W_h i W_γ są używane w każdym kroku czasowym sekwencji, niezależnie od jej długości. Dzięki temu sieć ma znacznie mniej parametrów do nauczenia niż gdyby każdy krok miał własne wagi, a wzorce wyuczone na jednej pozycji w sekwencji są automatycznie przenoszone na wszystkie inne pozycje. Wadą tego podejścia jest to, że jeden zestaw wag musi obsłużyć wszystkie kroki sekwencji, a wielokrotne mnożenie przez tę samą macierz W_h prowadzi bezpośrednio do problemów z zanikającym i eksplodującym gradientem.

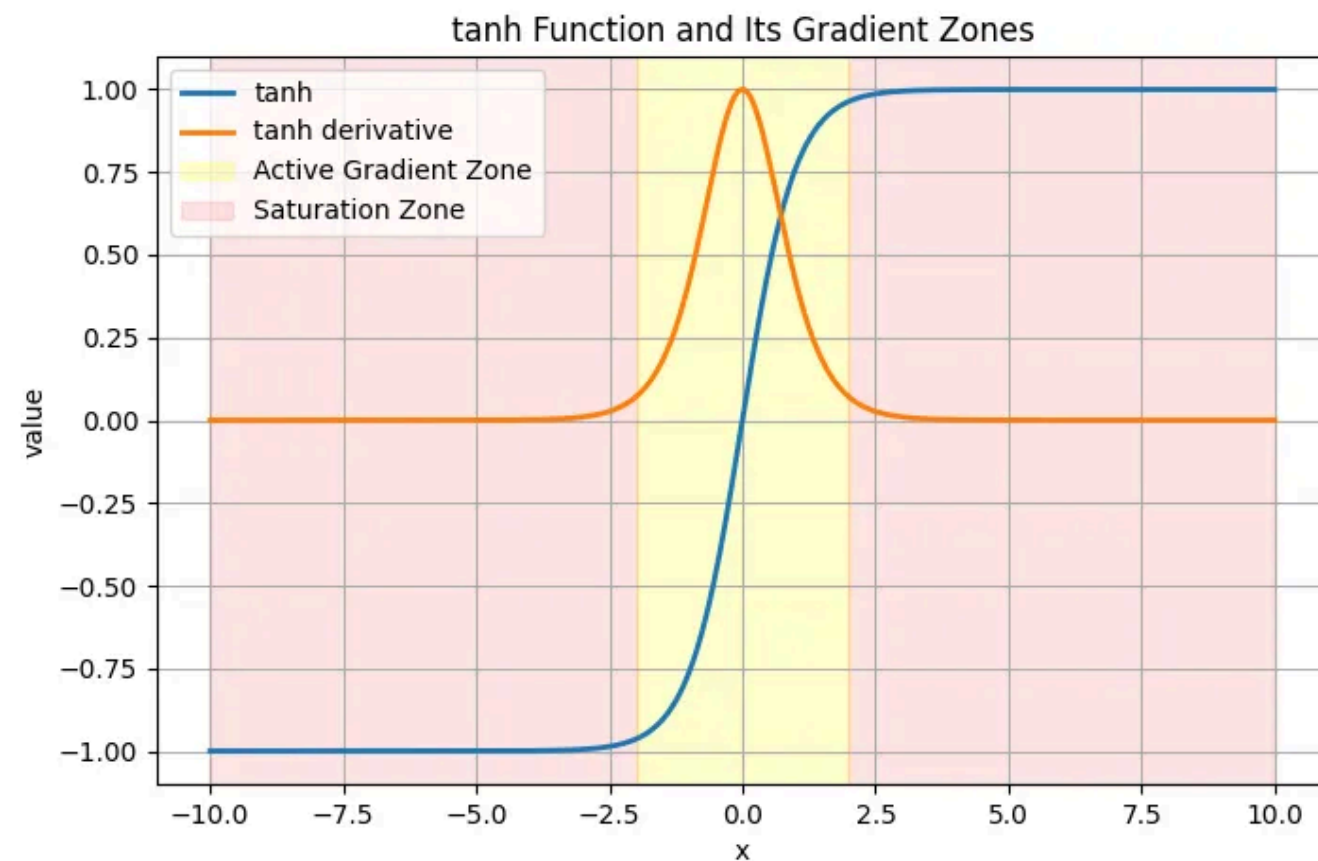
$$w = w - \eta \frac{\delta L}{\delta w}$$

η - learning rate

$$\theta = \theta - \eta \nabla J(\theta)$$

Sieci RNN są uczone algorytmem **Backpropagation Through Time** (BPTT), który jest rozszerzeniem klasycznej wstecznej propagacji błędu na sekwencje. Sieć jest najpierw "rozwijana w czasie" (sieć traktowana jest jak głęboka sieć, gdzie każdy krok czasowy to osobna warstwa) a następnie gradient błędu jest propagowany wstecz przez wszystkie kroki. Kluczową cechą uczenia RNN jest to, że te same wagi W_x , W_h i W_y są współdzielone przez wszystkie kroki czasowe, więc aktualizacja wag odbywa się przez zsumowanie gradientów ze wszystkich kroków sekwencji. Głównym problemem podczas uczenia są zanikające i eksplodujące gradienty, które wynikają z wielokrotnego mnożenia przez macierz wag rekurencyjnych W_h .

Gradient clipping jest stosowany do rozwiązywania problemu eksplodującego gradientu. Niestety problemu zanikającego gradientu nie da się tak łatwo rozwiązać.



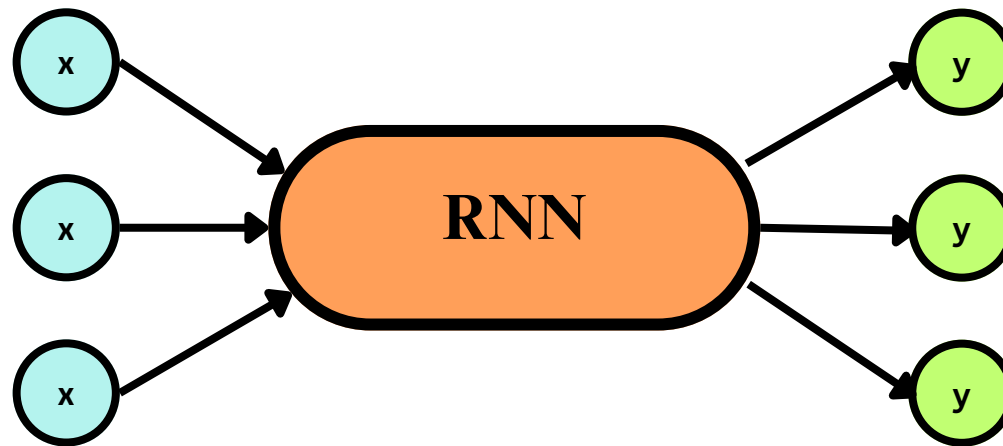
<https://medium.com/data-science-collective/a-visual-proof-and-explanation-of-how-the-activation-functions-influence-the-model-performance-2525d82460fd>

W sieciach RNN standardowo stosuje się funkcję aktywacji tanh zamiast ReLU z kilku powodów. Tanh ma zakres $(-1, 1)$ i jest wyśrodkowana przy zerze, co oznacza że gradienty mogą być zarówno dodatnie jak i ujemne, przez co sieć uczy się szybciej i stabilniej. ReLU natomiast daje tylko wartości nieujemne. Kolejnym powodem jest ograniczony zakres tanh - bez górnego limitu ReLU pozwala hidden state rosnać w nieskończoność przez kolejne kroki rekurencji, co destabilizuje trening. W zwykłej sieci ReLU jest stosowane raz na warstwę, natomiast w RNN ta sama macierz wag rekurencyjnych W_h jest aplikowana dziesiątki lub setki razy w pętli, co z ReLU bardzo łatwo prowadzi do eksplodujących wartości aktywacji.

Typy architektury RNN

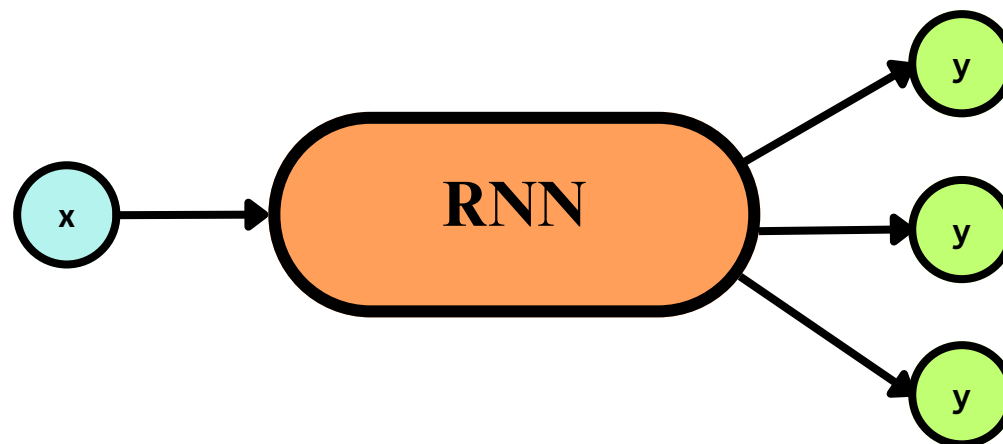


Wydział
Biologii



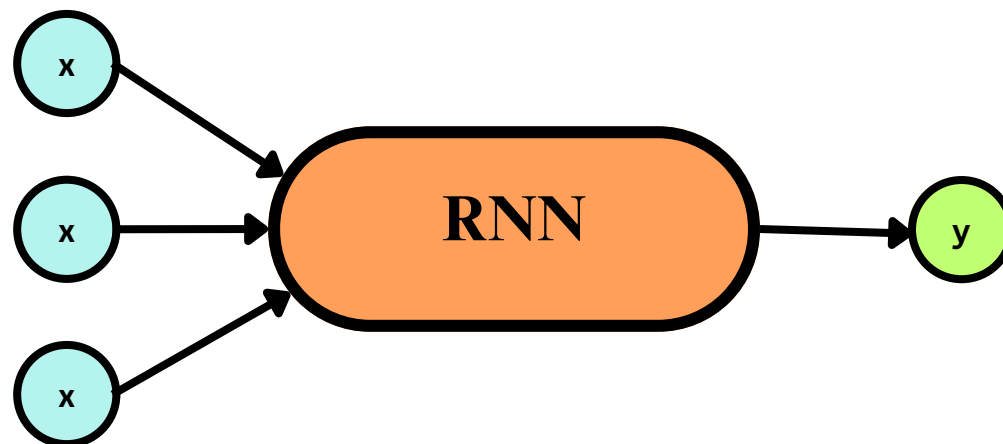
- **Many to Many**

np. tłumaczenie maszynowe, predykcja dalszych losów, opis video, rozpoznawanie mowy.



- **One to Many**

np. opis słowny obrazu, generowanie muzyki z nuty.



- **Many to One**

np. etykietowanie tekstu, klasyfikacja, wykrywanie spamu.